

HopperAudit

Software Requirements Specification

AI-Powered Waste Transfer Station Quality Gate

Version	1.0 — Hackathon MVP
Stack	Expo • FastAPI • PostgreSQL • Docker • AWS
AI	Computer Vision / Object Detection
AWS	S3 • ECS/Fargate • SQS • RDS • ECR • ALB • CloudWatch
Track	Waste & Energy

1. Executive Summary

HopperAudit is an AI-assisted quality-control system for neighborhood waste-transfer stations. A supervisor captures one or more views of a waste load using an Expo mobile application. Computer vision analyzes the media, contamination is scored, and the system returns PASS, HOLD, or REJECT with visual evidence.

Core workflow: capture → analyze → decide → record.

2. Problem & Objectives

Contamination can enter an organic waste stream before it is compacted and transported to downstream processing. HopperAudit adds an AI-assisted quality-control checkpoint at the transfer hopper.

- Fast mobile inspection workflow.
- Detect visible plastic, e-waste, batteries and other contamination.
- Provide explainable PASS / HOLD / REJECT decisions.
- Preserve annotated evidence and inspection history.
- Deploy a working Dockerized architecture on AWS.

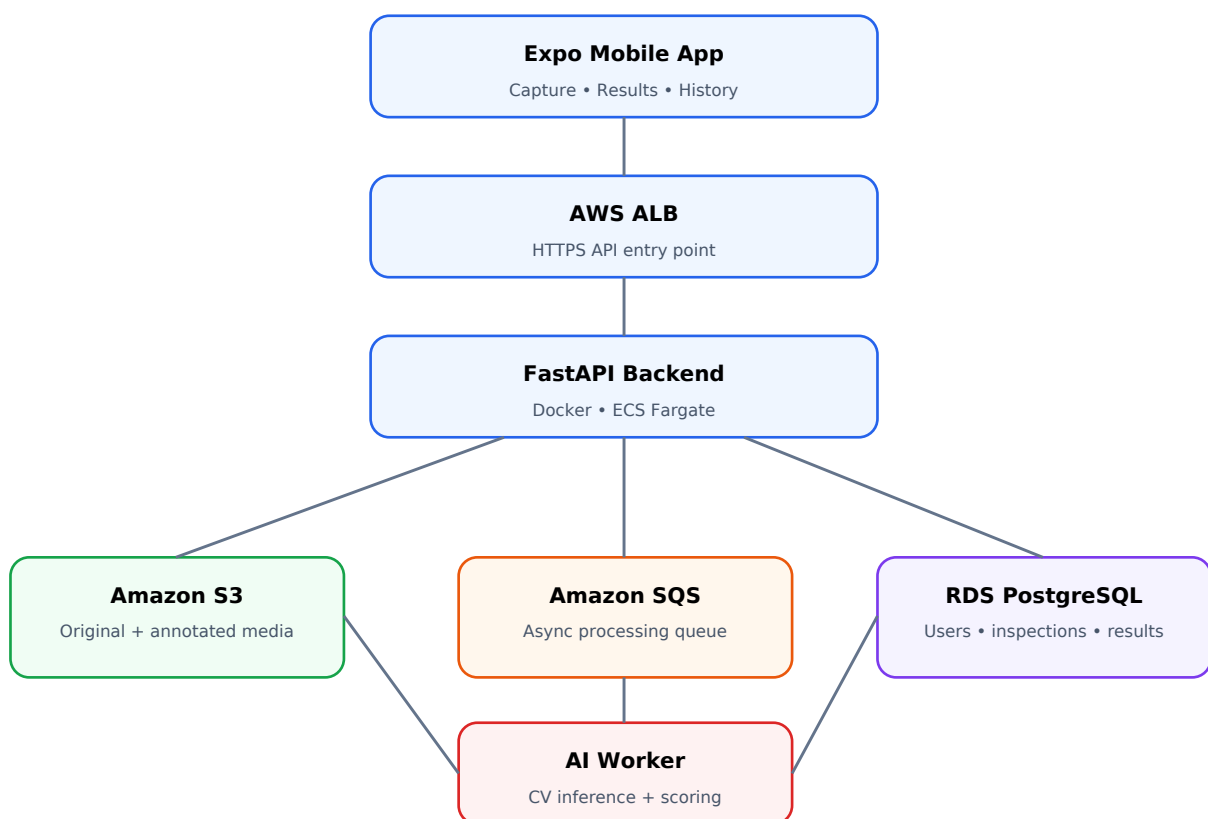
3. Scope

In scope: authentication, station selection, 1-5 image capture, upload, asynchronous analysis, object detection, contamination scoring, decisioning, annotated evidence, history, statistics, and AWS deployment.

Optional: short video upload with representative-frame analysis. Out of scope: robotic sorting, conveyor control, municipal ERP integration, payments, and continuous live inference.

4. System Architecture

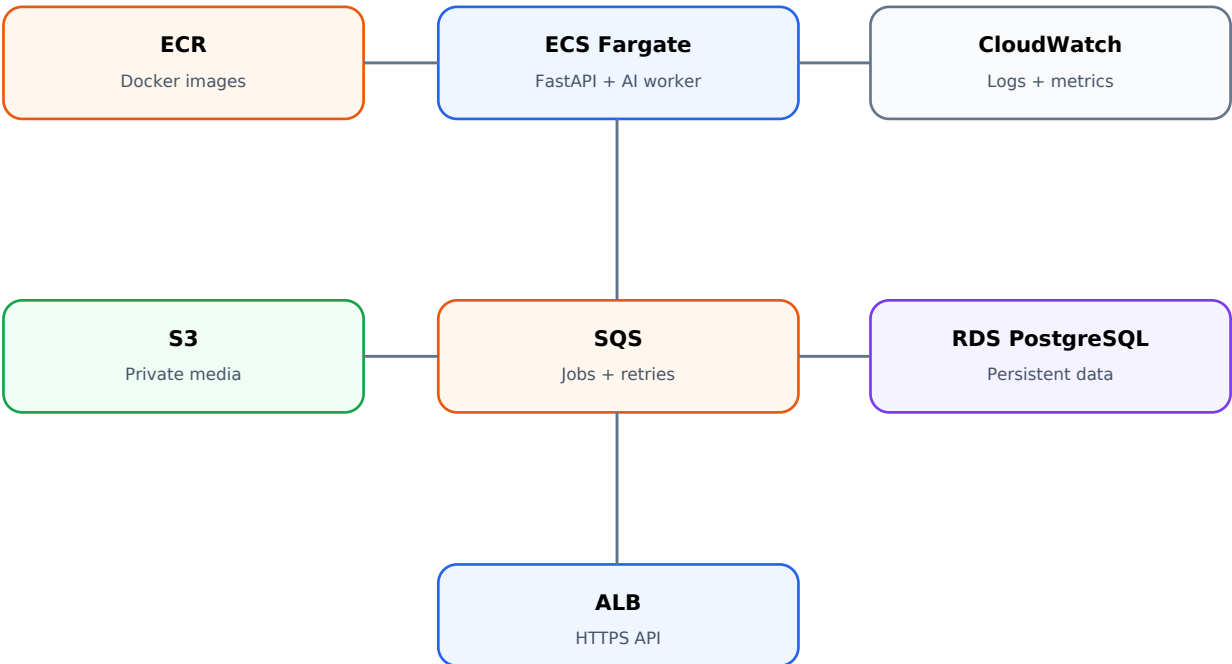
HOPPERAUDIT — SYSTEM / AWS ARCHITECTURE



Flow: Expo → S3 → FastAPI → SQS → AI Worker → PostgreSQL → Expo.

5. AWS Deployment Architecture

HOPPERAUDIT — AWS DEPLOYMENT



ECR stores Docker images; ECS/Fargate runs API and AI workers; S3 stores private media; SQS decouples processing; RDS PostgreSQL stores application data; ALB exposes HTTPS; CloudWatch provides logs and monitoring.

6. Functional Requirements

ID	Requirement	Description
FR-001	Authentication	Authorized supervisors can sign in and access protected APIs.
FR-002	Station Selection	Every inspection is associated with a transfer station.
FR-003	Create Inspection	Create unique ID and record user, station, timestamp and status.
FR-004	Capture Images	Capture 1-5 images and guide the user through useful views.
FR-005	Media Upload	Validate media and store evidence in private S3.
FR-006	Processing State	CREATED → UPLOADING → PROCESSING → COMPLETED/FAILED.
FR-007	Object Detection	Detect configured contamination categories.
FR-008	Confidence	Store confidence for every detection.
FR-009	Evidence	Generate annotated images with boxes, labels and confidence.
FR-010	Aggregation	Combine detections across multiple images.
FR-011	Contamination Score	Calculate configurable inspection-level score.
FR-012	Decision	Return PASS, HOLD or REJECT.
FR-013	Hazard Override	High-risk detections may force REJECT.
FR-014	Result Screen	Show decision, score, detections, evidence and processing time.
FR-015	History	Browse previous inspections.
FR-016	Details	View original/annotated media and complete result.
FR-017	Statistics	Show totals, decision counts and contamination rate.
FR-018	Video (Optional)	Accept short video and analyze representative frames asynchronously.

7. AI Processing Requirements

Pipeline

Media → preprocessing → object detection → confidence filtering → severity assignment → multi-image aggregation → contamination score → PASS/HOLD/REJECT → annotated evidence

Initial classes

- Plastic film
- Plastic bag
- PET/plastic container
- Battery
- E-waste
- Glass
- Metal
- Sanitary/hazardous material

Example scoring

Illustrative defaults: clean material +100; plastic film –20; plastic bag –15; glass –10; e-waste –40; battery –60; hazardous material –80. Example thresholds: ≥85 PASS, 60–84 HOLD, <60 REJECT. These are product defaults, not regulatory limits.

8. Data Model

Table	Key fields
users	id, name, email, password_hash, role, station_id, created_at
stations	id, name, ward, address, latitude, longitude, created_at
inspections	id, station_id, user_id, status, decision, contamination_score, processing_time_ms, created_at, completed_at
media	id, inspection_id, type, original_url, annotated_url, created_at
detections	id, media_id, category, confidence, x, y, width, height, severity, created_at

9. API Requirements

POST /api/v1/inspections

Creates an inspection. Example: {"inspection_id":"INS-1042","status":"CREATED"}

POST /api/v1/inspections/{id}/media

Uploads/registers media and returns media count and status.

POST /api/v1/inspections/{id}/analyze

Queues AI processing. Example: {"inspection_id":"INS-1042","status":"PROCESSING"}

GET /api/v1/inspections/{id}

Returns status, decision, score, detections and evidence.

GET /api/v1/inspections

Returns paginated history with optional filters.

GET /api/v1/dashboard/stats

Returns station-level inspection statistics.

10. Non-Functional Requirements

- Target image processing: approximately 2-5 seconds depending on model and deployment resources.
- Minimal supervisor interaction: capture → submit → review.
- Failures must be explicit and retryable.
- Use HTTPS, authentication, authorization, private DB access, secret management and controlled S3 access.
- API and AI workers should scale independently through asynchronous processing.
- CloudWatch logging should cover requests, processing duration and errors.

11. Security & Privacy

- HTTPS for client-to-API communication.
- Authenticated and authorized protected endpoints.
- No credentials in source control.
- Private PostgreSQL access.
- Private S3 objects accessed through controlled URLs/backend authorization.
- Minimize personally identifiable information.

12. MVP Acceptance Criteria

The complete path must work:

Expo → capture 1-3 images → upload → create inspection → queue AI job → analyze → detect contamination → generate evidence → calculate score → PASS/HOLD/REJECT → display result → persist PostgreSQL record → show history.

13. Demonstration Scenario

Input	3 images of one waste load containing organic material and visible contamination.
AI	Detect battery + plastic contamination and produce confidence scores.
Decision	REJECT or HOLD based on configured thresholds.
Evidence	Annotated image with bounding boxes and labels.
Persistence	Inspection saved in PostgreSQL; media stored in S3.

14. Delivery Priorities

Must work: Expo capture, FastAPI, AI inference, PASS/HOLD/REJECT, evidence, PostgreSQL persistence.

AWS proof: S3 + ECS/Fargate + RDS + SQS + ECR + CloudWatch.

Demo polish: clear result screen, annotated evidence, history and architecture explanation.

Stretch: short-video processing and analytics dashboard.

15. Future Roadmap

- Short video with representative-frame sampling.
- Ward/route/contractor contamination trends.
- Model improvement from reviewed inspection data.
- Offline capture and synchronization.
- Live camera-assisted inspection.
- Municipal processing-facility integration.

End of HopperAudit SRS v1.0